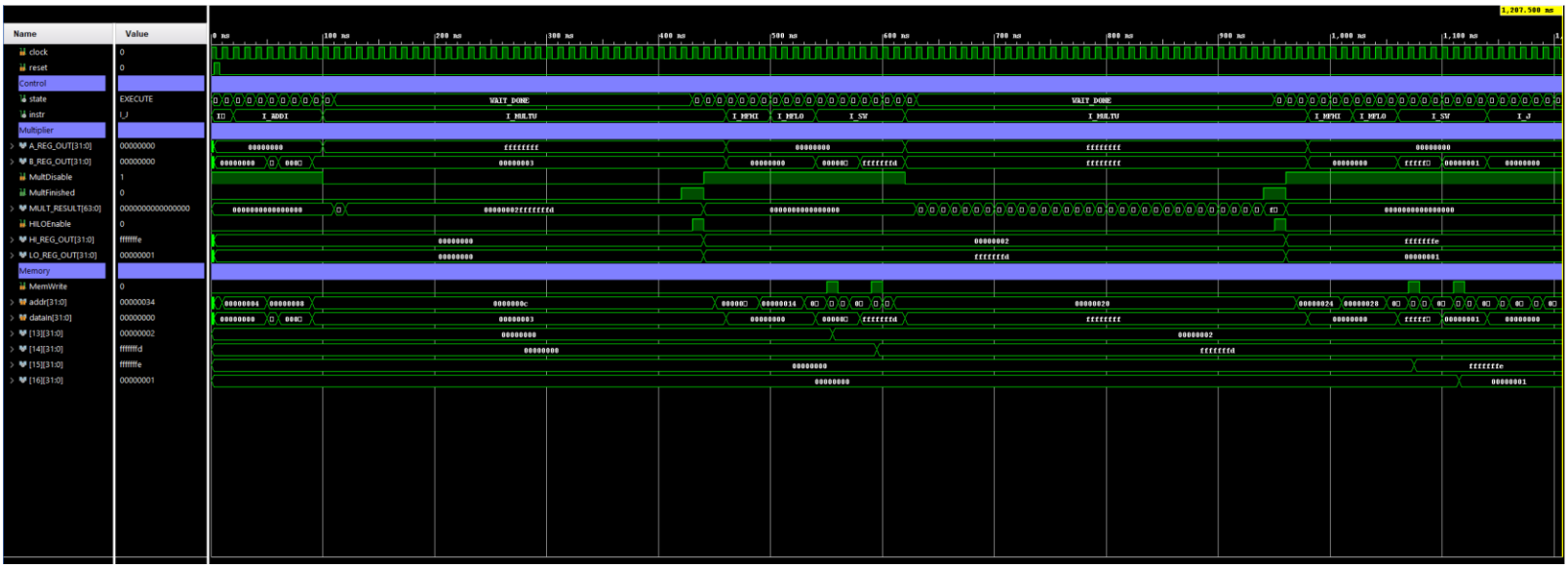


(image is nearly illegible; also at https://fs.emahizer.net/upload/waveform.png)

Assembly script:

```
addi  $1, $0, -1      # 0xFFFFFFFF
addi  $2, $0, 3
multu $1, $2          # 0x00000002_FFFFFFFD
mfhi  $3
mflo  $4
sw    $3, 52($0)      # word 12
sw    $4, 56($0)      # word 13
multu $1, $1          # 0xFFFFFFFE_00000001: checks a second MULTU restarts cleanly
mfhi  $5
mflo  $6
sw    $5, 60($0)      # word 14
sw    $6, 64($0)      # word 15
done: J done
```

Tcl script:

```
# restart the simulation
restart

# top-level CPU testbench is named cpu_tb; U_0 is the CPU and U_1 is the memory block
add_wave {{/cpu_tb/clock}}
add_wave {{/cpu_tb/reset}}

add_wave_divider "Control"
add_wave {{/cpu_tb/U_0/control/state}}
```

```
add_wave {{/cpu_tb/U_0/control/instr}}

add_wave_divider "Multiplier"
add_wave -radix hex {{/cpu_tb/U_0/datapath/A_REG_OUT}}
add_wave -radix hex {{/cpu_tb/U_0/datapath/B_REG_OUT}}
add_wave {{/cpu_tb/U_0/datapath/MultDisable}}
add_wave {{/cpu_tb/U_0/datapath/MultFinished}}
add_wave -radix hex {{/cpu_tb/U_0/datapath/MULT_RESULT}}
add_wave {{/cpu_tb/U_0/datapath/HILOEnable}}
add_wave -radix hex {{/cpu_tb/U_0/datapath/HI_REG_OUT}}
add_wave -radix hex {{/cpu_tb/U_0/datapath/LO_REG_OUT}}

add_wave_divider "Memory"
add_wave {{/cpu_tb/U_1/MemWrite}}
add_wave -radix hex {{/cpu_tb/U_1/addr}}
add_wave -radix hex {{/cpu_tb/U_1/dataIn}}
add_wave -radix hex {{/cpu_tb/U_1/mw_U_0ram_table[13]}}
add_wave -radix hex {{/cpu_tb/U_1/mw_U_0ram_table[14]}}
add_wave -radix hex {{/cpu_tb/U_1/mw_U_0ram_table[15]}}
add_wave -radix hex {{/cpu_tb/U_1/mw_U_0ram_table[16]}}

# Multiplication test program (instruction memory)
#
# addi  $1, $0, -1      # $1 = 0xFFFFFFFF (4294967295 unsigned)
# addi  $2, $0, 3       # $2 = 3
# multu $1, $2          # HI:LO = 0x00000002_FFFFFFFD (unsigned)
# mfhi  $3
# mflo  $4
# sw    $3, 52($0)      # word 12
# sw    $4, 56($0)      # word 13
# multu $1, $1          # HI:LO = 0xFFFFFFFE_00000001 (second MULTU: multiplier restarts)
# mfhi  $5
# mflo  $6
# sw    $5, 60($0)      # word 14
# sw    $6, 64($0)      # word 15
# done: j done

# Words 13-16 hold the results, so they are not forced; a forced word can't be stored to.
add_force {/cpu_tb/U_1/mw_U_0ram_table[0]}  -radix hex {2001FFFF}
add_force {/cpu_tb/U_1/mw_U_0ram_table[1]}  -radix hex {20020003}
add_force {/cpu_tb/U_1/mw_U_0ram_table[2]}  -radix hex {00220019}
add_force {/cpu_tb/U_1/mw_U_0ram_table[3]}  -radix hex {00001810}
add_force {/cpu_tb/U_1/mw_U_0ram_table[4]}  -radix hex {00002012}
add_force {/cpu_tb/U_1/mw_U_0ram_table[5]}  -radix hex {AC030034}
```

```
add_force {/cpu_tb/U_1/mw_U_0ram_table[6]}  -radix hex {AC040038}
add_force {/cpu_tb/U_1/mw_U_0ram_table[7]}  -radix hex {00210019}
add_force {/cpu_tb/U_1/mw_U_0ram_table[8]}  -radix hex {00002810}
add_force {/cpu_tb/U_1/mw_U_0ram_table[9]}  -radix hex {00003012}
add_force {/cpu_tb/U_1/mw_U_0ram_table[10]} -radix hex {AC05003C}
add_force {/cpu_tb/U_1/mw_U_0ram_table[11]} -radix hex {AC060040}
add_force {/cpu_tb/U_1/mw_U_0ram_table[12]} -radix hex {0800000C}

add_force clock 1 {0 5ns} -repeat_every 10ns
add_force reset 0
run 2500ps
add_force reset 1
run 5 ns
add_force reset 0

# two multiplications at ~35 cycles each, plus the other instructions: ~110 cycles
run 1200 ns

# check results (same style as the provided test script)
if {[get_value -radix unsigned {/cpu_tb/U_1/mw_U_0ram_table[13]}] == 0x00000002} {
   puts "Correct! MFHI after 0xFFFFFFFF x 3 passed."
} else {
   puts "Incorrect! MFHI after 0xFFFFFFFF x 3 didn't pass."
}
if {[get_value -radix unsigned {/cpu_tb/U_1/mw_U_0ram_table[14]}] == 0xFFFFFFFD} {
   puts "Correct! MFLO after 0xFFFFFFFF x 3 passed."
} else {
   puts "Incorrect! MFLO after 0xFFFFFFFF x 3 didn't pass."
}
if {[get_value -radix unsigned {/cpu_tb/U_1/mw_U_0ram_table[15]}] == 0xFFFFFFFE} {
   puts "Correct! MFHI after 0xFFFFFFFF x 0xFFFFFFFF passed."
} else {
   puts "Incorrect! MFHI after 0xFFFFFFFF x 0xFFFFFFFF didn't pass."
}
if {[get_value -radix unsigned {/cpu_tb/U_1/mw_U_0ram_table[16]}] == 0x00000001} {
   puts "Correct! MFLO after 0xFFFFFFFF x 0xFFFFFFFF passed."
} else {
   puts "Incorrect! MFLO after 0xFFFFFFFF x 0xFFFFFFFF didn't pass."
}
```

Explanation:

The multiplication instruction worked perfectly in Tcl the first time for me; I properly had it enter the correct states etc, so it just kinda worked. It did not work in the block diagram immediately, though; I had to change the RAM to be how the CPU_memory.vhd was; on falling edges of the clock, it reads/writes, but if it wasn't inverted (like the default block diagram), it failed to write correctly for some parts of the memory (like word 0; it would fail to write the top half in the C code.) Flipping the clock fixed it. All of my C++ tests also worked (with edge cases + random numbers; included in my .zip).